

Content-Based Filtering

This approach differs from collaborative filtering by explicitly using the attributes (features) of users and items to find a good match.

Collaborative vs. Content-Based

- **Collaborative Filtering:** Recommends items based on the behavior of *similar users* (e.g., "People who bought X also bought Y"). It does not need to know *what* the item is.
 - **Content-Based Filtering:** Recommends items based on the match between a user's *profile* and an item's *features*. It requires specific features for both.
-

Feature Vectors

1. User Features ($x_u^{(j)}$)

We construct a feature vector for user j based on what we know about them.

- **Demographics:** Age, Gender (one-hot), Country (one-hot).
- **Behavior:** Which of the top 1,000 movies have they watched?
- **Derived Preferences:** What is their average rating for "Romance" movies? What is their average rating for "Action" movies?
 - *Note:* It is perfectly fine to build user features based on their past ratings.

2. Movie (Item) Features ($x_m^{(i)}$)

We construct a feature vector for movie i .

- **Attributes:** Release year, Genre (one-hot), Cast, Studio.
- **Reviews:** Features derived from critic reviews.
- **Derived Stats:** Average rating of this movie.

Key Difference: The user feature vector (x_u) and the movie feature vector (x_m) can have **completely different sizes**. (e.g., user vector length = 1500, movie vector length = 50).

The Matching Concept: Dot Products

In Collaborative Filtering, we predicted a rating as $w^{(j)} \cdot x^{(i)} + b$.

In Content-Based Filtering, we want to predict the rating by taking the **dot product of two vectors**: a "User Vector" (v_u) and a "Movie Vector" (v_m).

$$\text{Prediction} = v_u^{(j)} \cdot v_m^{(i)}$$

- $v_u^{(j)}$: A vector representing the user's preferences (computed from user features x_u).
- $v_m^{(i)}$: A vector representing the movie's attributes (computed from movie features x_m).

Crucial Requirement:

Even though the raw inputs x_u and x_m have different sizes, the computed vectors v_u and v_m **must be the same size** (e.g., both are length 32) so we can take their dot product.

Example Interpretation:

- If $v_u = [4.9, 0.1]$ (User loves Romance, hates Action).
 - If $v_m = [4.5, 0.2]$ (Movie is highly Romance, low Action).
 - The dot product will be high, indicating a good match.
-

The Goal

The challenge of Content-Based Filtering is: **How do we turn the raw features (x_u, x_m) into these compact, matching vectors (v_u, v_m)?**